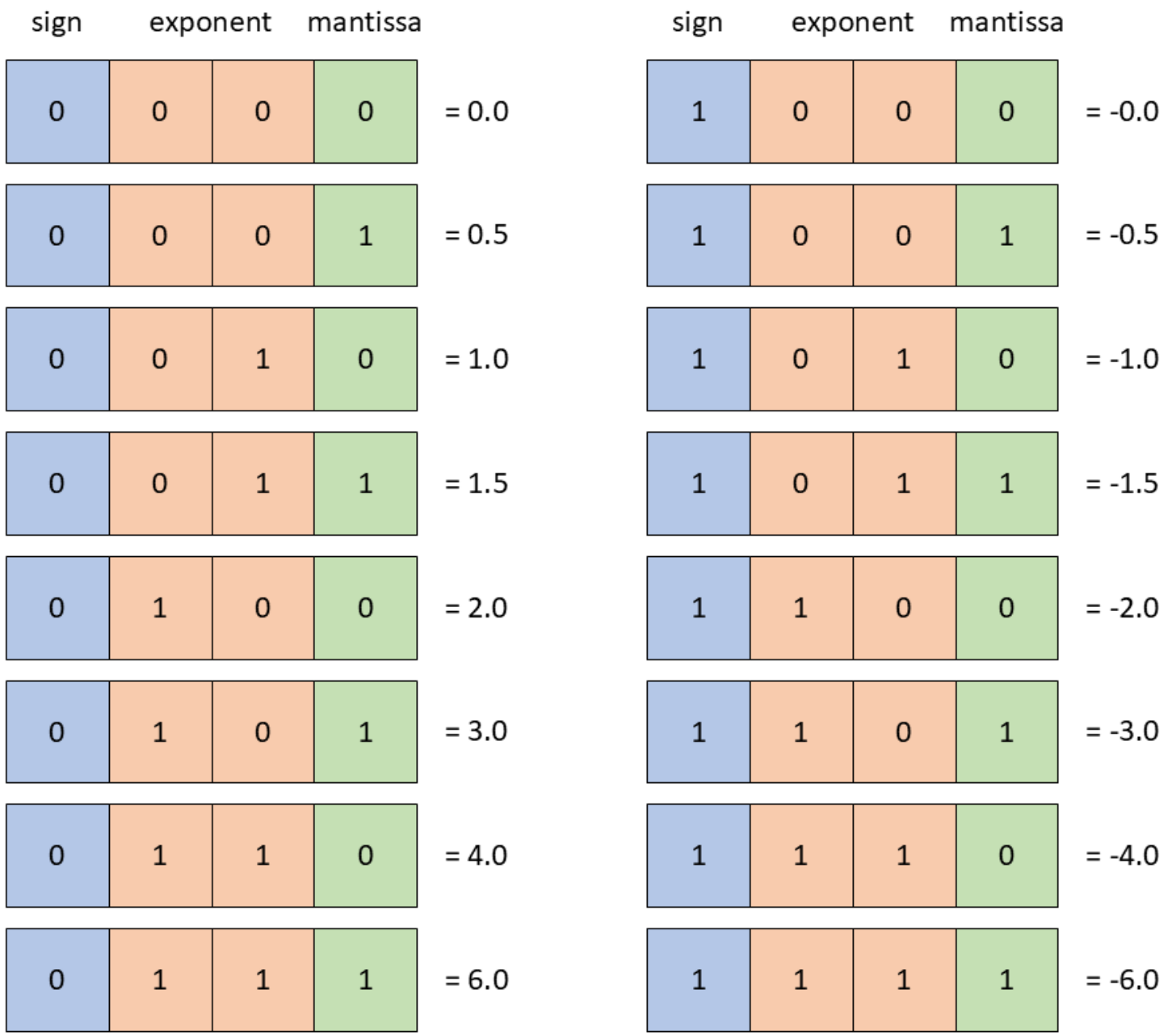
NVIDIA FP4 and Scaling Factor

Sparsh Mittal

FP4

FP4 E2M1 format can represent values of magnitude up to +/-6.

Challenge: Range is so small! How to maintain numerical accuracy across a wide dynamic range of tensor values?

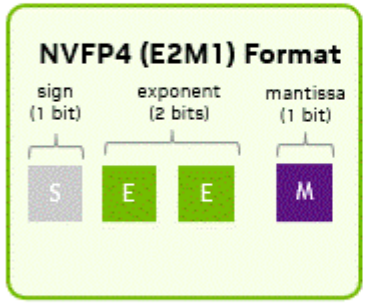


Feature	FP4 (E2M1)	MXFP4	NVFP4
Format Structure	4 bits (1 sign, 2 exponent, 1 mantissa) plus software scaling factor	4 bits (1 sign, 2 exponent, 1 mantissa) plus 1 shared power-of-two (E8M0) scale per 32 value block	4 bits (1 sign, 2 exponent, 1 mantissa) plus 1 shared FP8 scale (E4M3) per 16 value block. Plus, FP32 scaling factor at tensor level
Accelerated Hardware Scaling	No	Yes	Yes
Memory	~25% of FP16		
Accuracy	Risk of noticeable accuracy drop compared to FP8	Risk of noticeable accuracy drop compared to FP8	Lower risk of noticeable accuracy drop particularly for larger models

NVFP4

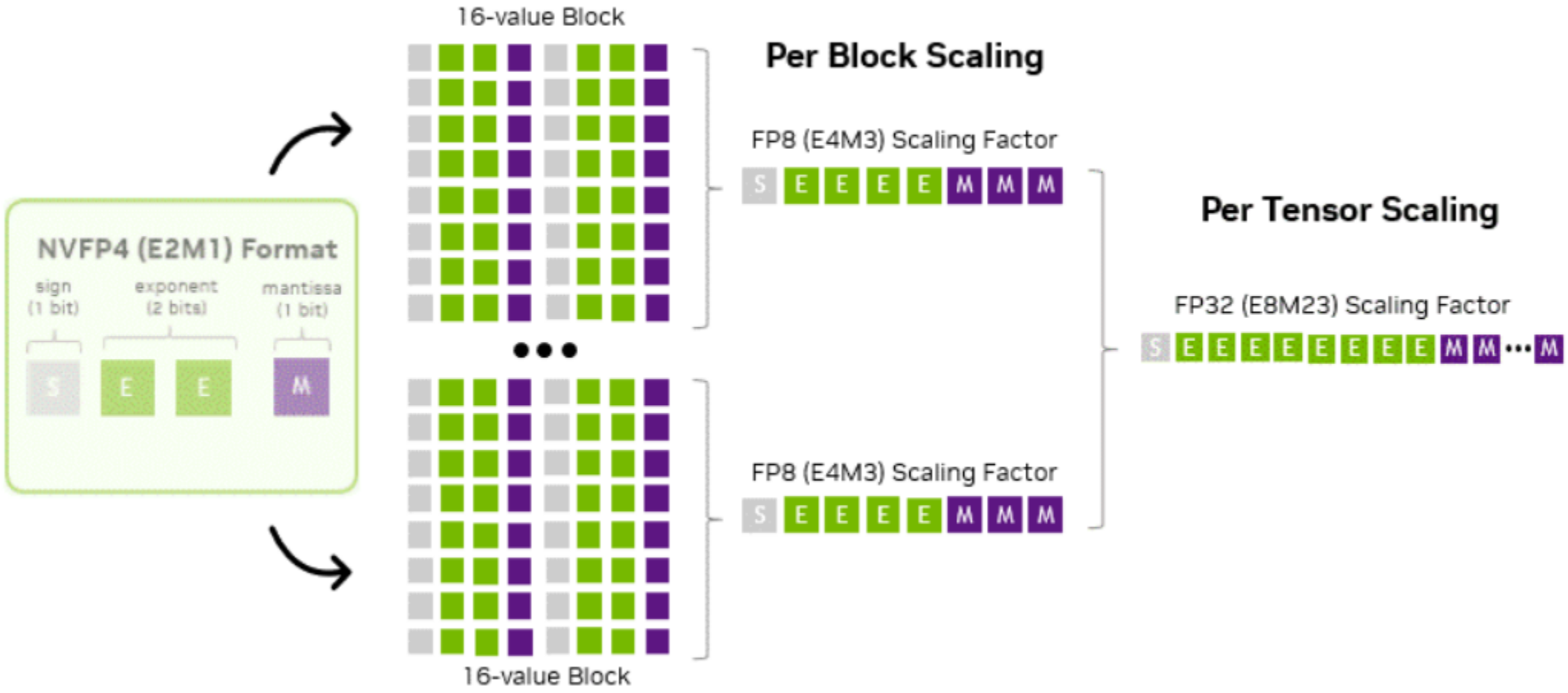
- Two ideas:
  - High-precision scale encoding
  - A two-level micro-block scaling strategy
- This strategy applies
  - a fine-grained E4M3 scaling factor to each 16-value *micro-block*, a compact subset of the larger tensor,
  - and a second-level FP32 scalar applied per tensor.
- Together, these two levels of scaling enable more accurate value representation and significantly reduce quantization error

NVFP4 two-level scaling per-block and per-tensor precision structure



This is a gif, so not visible in PDF. See next slide for the same figure in png.

NVFP4 two-level scaling per-block and per-tensor precision structure



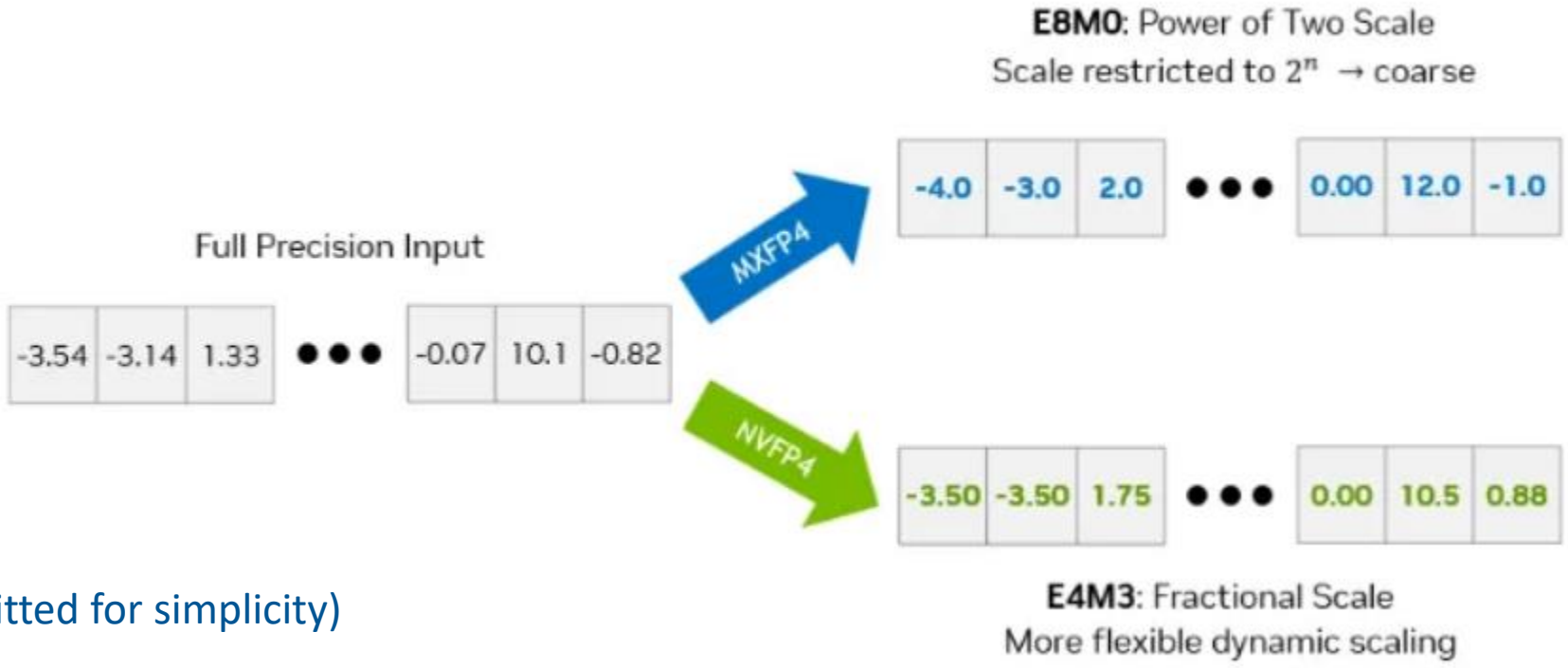
How to get value out of the shared micro-block scaling?

NVFP4 encodes blocks using E4M3 FP8 precision scaling factor.

NVFP4 uses the E4M3 FP8 format variant that enables non-power-of-two scaling factors with fractional precision.

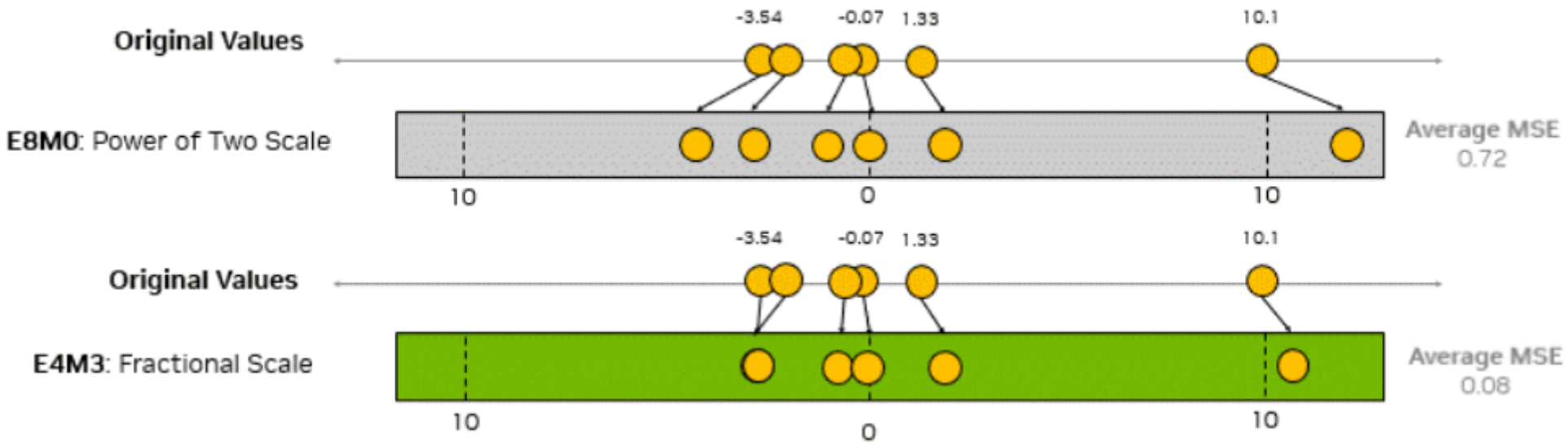
This added flexibility enables more accurate encoding of the tensor's actual distribution.

An example of a full precision input matrix and the resulting quantized matrices using E8M0 and E4M3 scaling.



(Second-level FP32 scaling omitted for simplicity)

Comparison of quantization error when encoding scaling factors with E8M0 versus E4M3 (second-level FP32 scaling omitted for simplicity)



What makes E4M3 “better on average” is that it picks that one fractional scale so that, when the squared (or absolute) errors over all 16 values are summed, the total error is generally smaller than an E8M0-quantized block

E8M0 vs E4M3 scaling

E8M0 = Snaps the scale factor to nearest 2^n

This can create a large quantization error for the block maximum and can often lead to larger overall quantization errors for blocks.

E4M3 = Finds one scale factor that makes the block errors collectively as small as possible.

This often improves accuracy for the block maximum—though some values might be slightly less accurate, the block as a whole retains higher fidelity.

Second-level scaling

The downside of having a more precise scaling factor with E4M3 is the reduced range of scale values.

This is counteracted by utilizing a second-level scaling factor.

This second-level scaling factor is done at a per-tensor level with FP32, which adjusts the original tensor's distribution such that the micro-blocks can be effectively encoded using E4M3 scale factors.

E8M0 vs E4M3

Pros of E8M0: E8M0 is very simple.

E8M0 scale factors reduce computational complexity because they don't require an extra per-tensor software scaling factor

They can be adequate for activations and weights that are less sensitive to the precision of scale factors.

Pros of E4M3: It adjusts its scaling factor to each small block of values, allowing for finer fit across wider ranges of inputs.

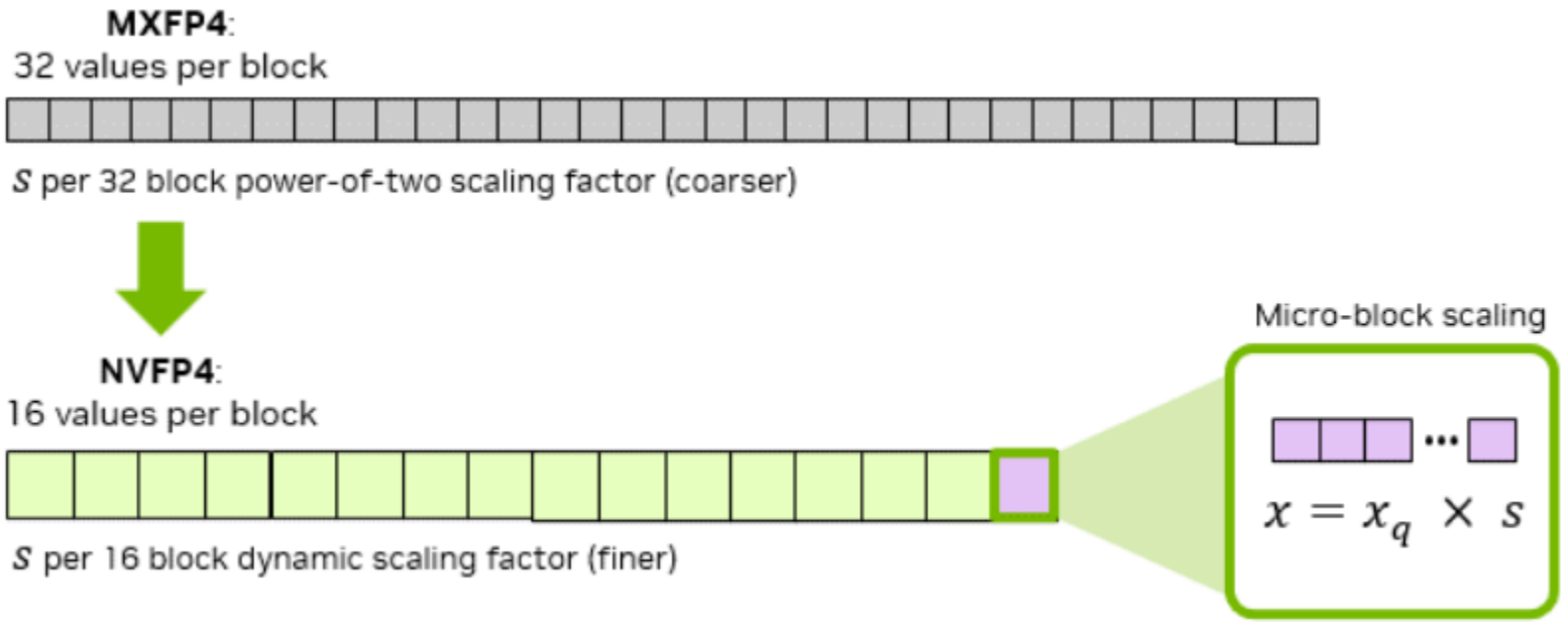
That additional flexibility is what translates to a lower overall rounding error and preservation of model intelligence when quantizing to 4-bits using NVFP4.

Another key component of NVFP4 is the block floating-point representation, where micro-blocks share a common scaling factor.

By reducing the group size from 32 elements to 16 values per block, NVFP4 enables finer-grained scaling than MXFP4.

Large tensors in AI models often mix large and small numbers, and a single “umbrella” scaling can lead to significant quantization errors that degrade model performance.

The tighter grouping in NVFP4 offers twice as many opportunities to match the local dynamic range of the data, significantly reducing those errors.


MXFP4:
32 values per block
S per 32 block power-of-two scaling factor (coarser)
NVFP4:
16 values per block
S per 16 block dynamic scaling factor (finer)
Micro-block scaling
x = x_q × s

Both formats rely on grouped value blocks and shared scale factors, but a key innovation in NVFP4 lies in its smaller block size and robust scaling.

By cutting the block size in half–from 32 values to 16–NVFP4 enables more localized adaptation to the data's dynamic range.

This makes it easier to preserve small-but-important differences in model weights or activations.

Inside each 16-value block, every 4-bit encoded value x_q (between the range of -6 to +6) is scaled using: $x = x_q * S$

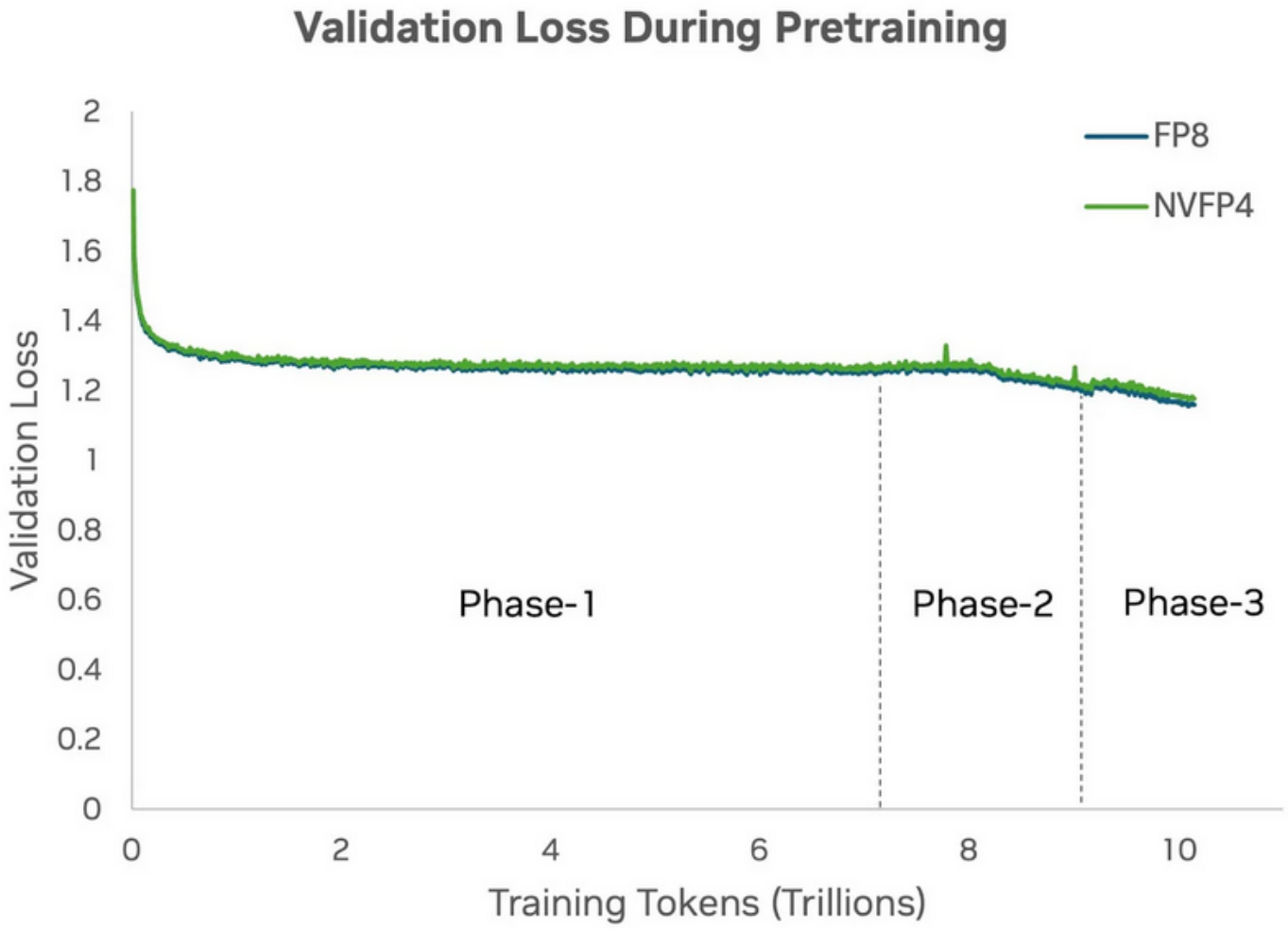
In this equation, S is a higher precision FP8 (E4M3) scale that's computed dynamically to minimize the overall block error.

By recomputing S for each group of 16 elements, NVFP4 minimizes quantization error at 4-bit precision, while still significantly reducing memory and compute complexity compared to higher-precision formats.

This structure makes NVFP4 not just a low-precision format, but one that makes major strides in preserving model intelligence

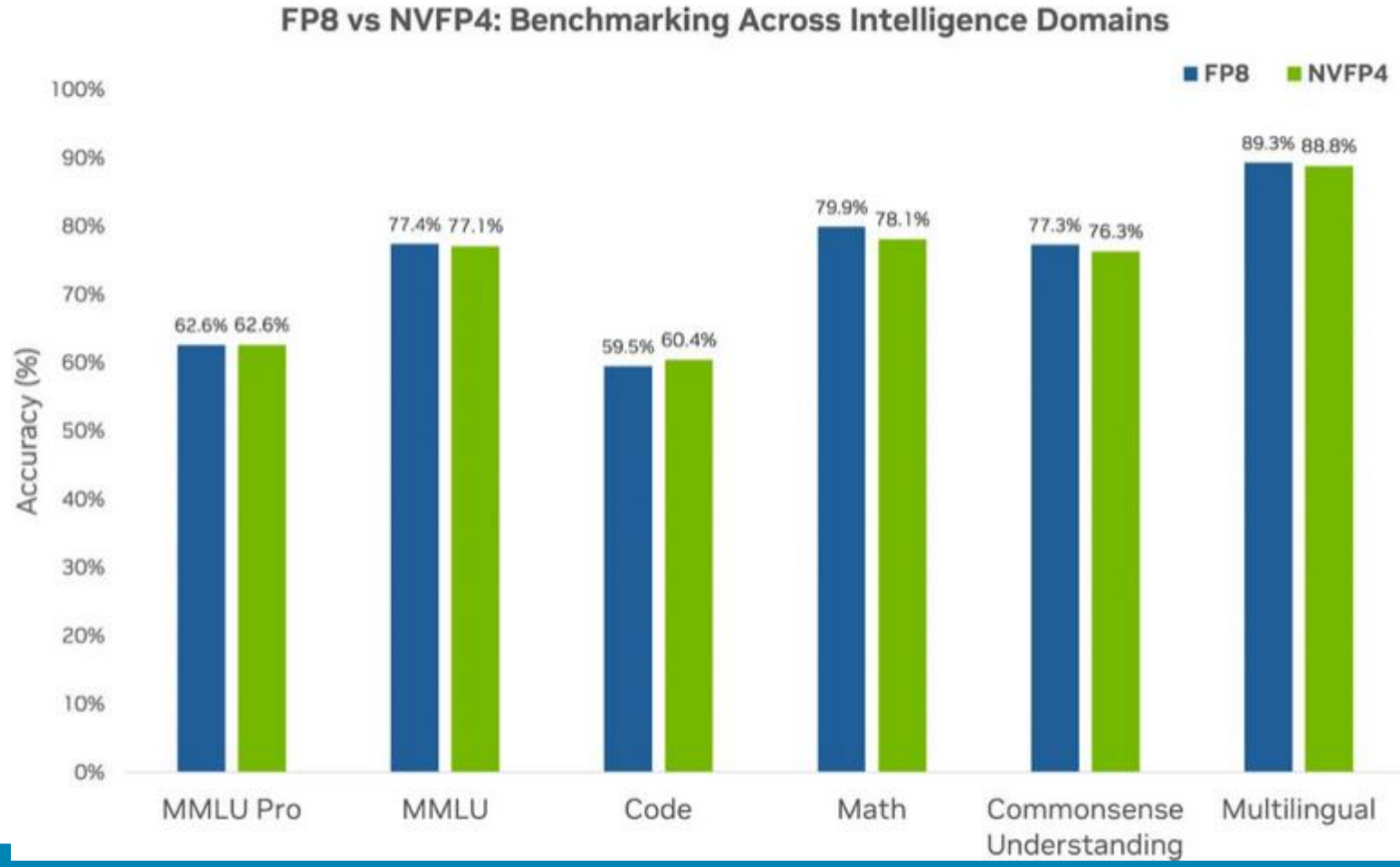
Validation loss comparison during pretraining of a 12B Hybrid Mamba-Transformer model using FP8 and NVFP4 precisions over 10T tokens

NVFP4 loss curve closely aligns with the FP8 loss curve (baseline) throughout training.

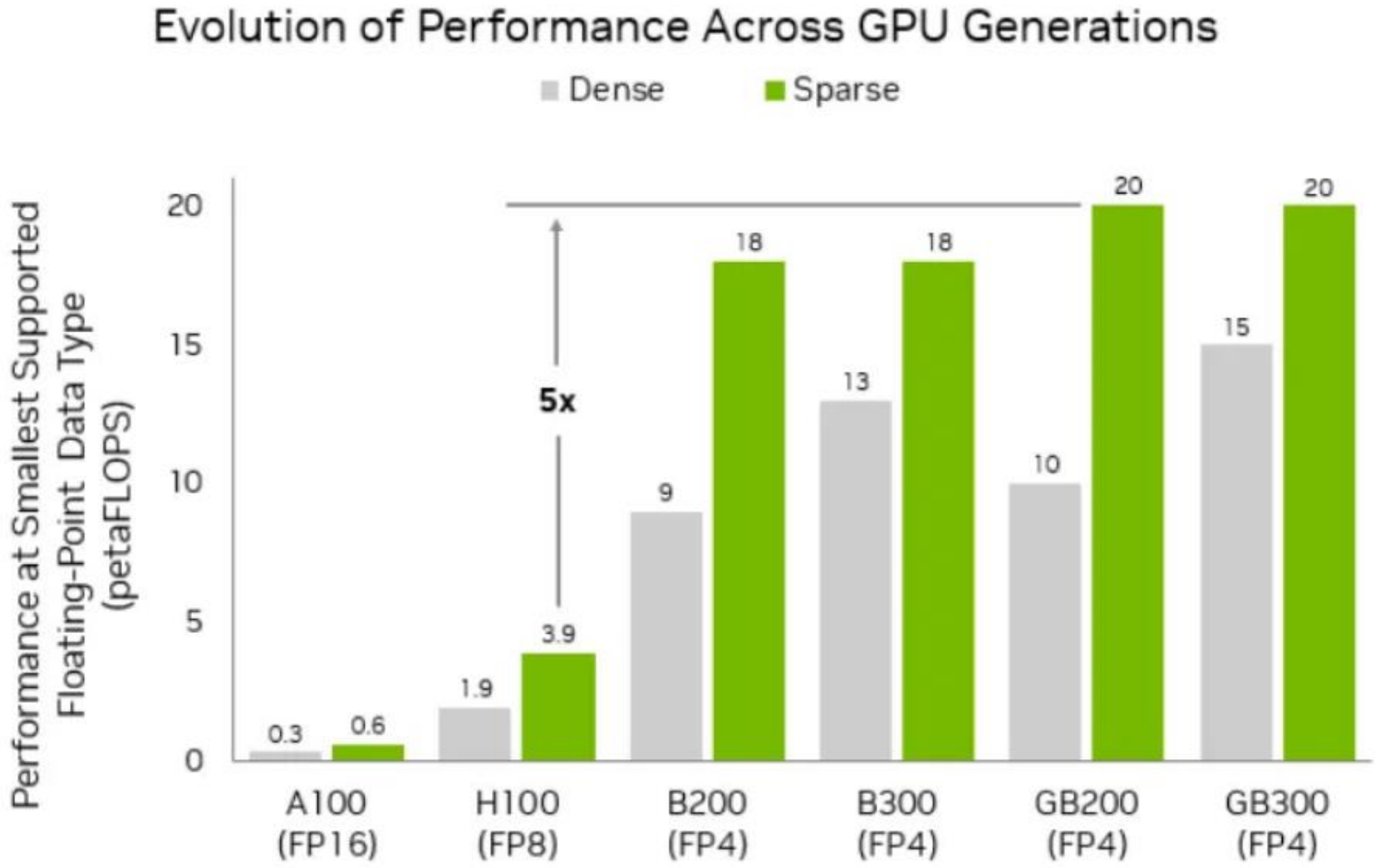


Benchmarking downstream task accuracy scores on pretraining the 12B Hybrid Mamba-Transformer model using FP8 precision (baseline) and NVFP4 precision.

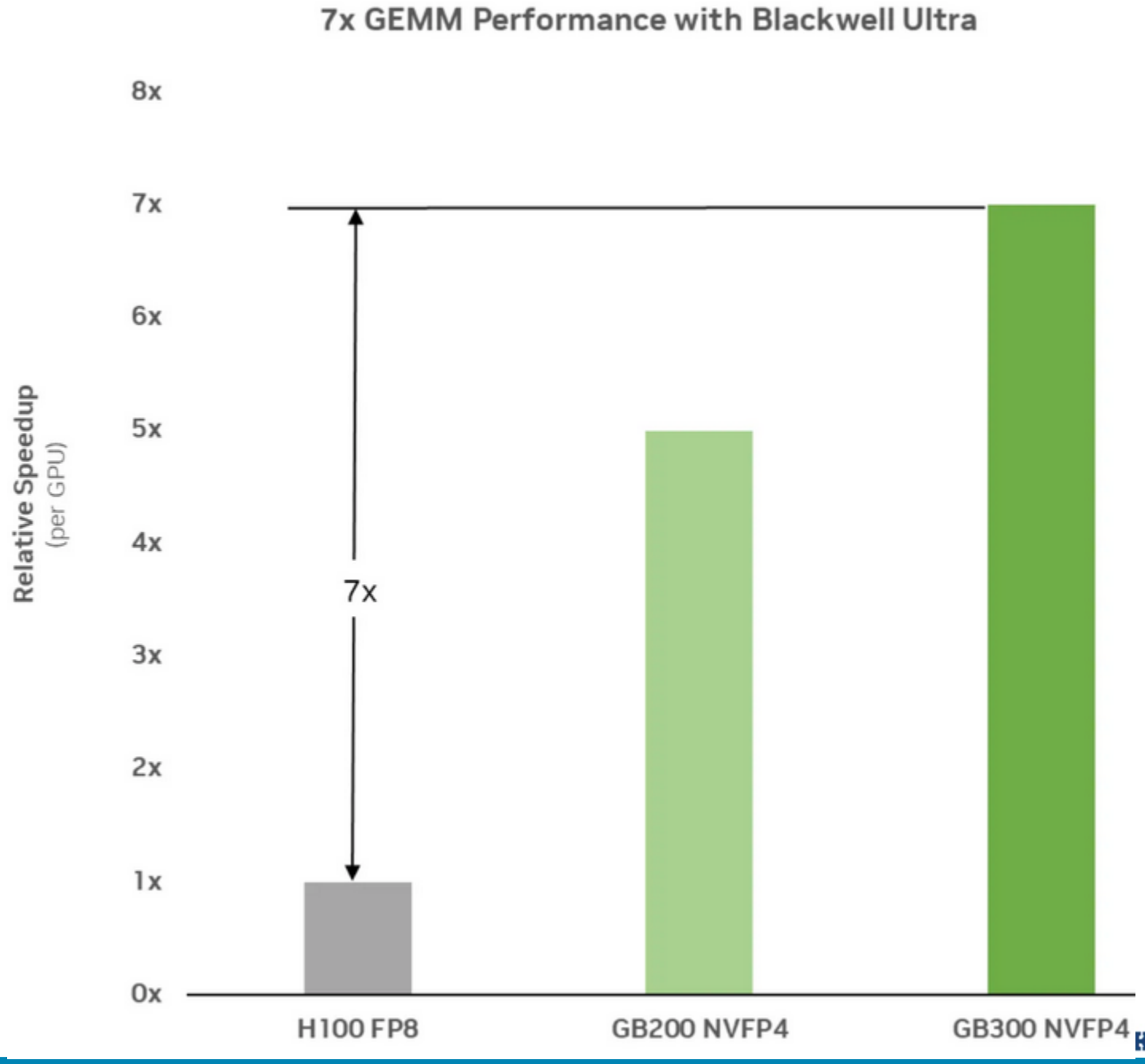
Pretraining with NVFP4 achieves accuracy comparable with higher precision formats.



Peak low-precision performance across NVIDIA GPU architectures



Measured GEMM performance shows GB300 delivering a 7x speedup over Hopper, accelerating core LLM training operations through faster FP4-optimized matrix multiplications.



https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/

https://developer.nvidia.com/blog/floating-point-8-an-introduction-to-efficient-lower-precision-ai-training/

https://developer.nvidia.com/blog/nvfp4-trains-with-precision-of-16-bit-and-speed-and-efficiency-of-4-bit/

https://docs.nvidia.com/deeplearning/transformer-engine/user-guide/examples/fp8_primer.html